



KTH Technology and Health

TENTAMEN

Kurs, kursnummer	HI1027, Objektorienterad programmering
Moment:	TEN1, 3,5 hp
Program: Åk:	TIDAA, TIELA, TIMEL -
Examinator:	Anders Lindström
Rättande lärare:	Anders Lindström
Datum: Tid:	2024-03-11 14:00 – 19:00
Hjälpmedel:	IntelliJ och Netbeans finns på tentamenskontot API-dokumentation och en kort syntaxlista finns i mappen P/Java/api/index.html
Omfattning och betygsgränser:	För betyget E krävs minst hälften av poängen på A-delen samt och hälften av poängen på B-delen samt att minst en av de två första programmeringsuppgifterna är helt korrekt lösta. B-delen rättas endast om minst hälften av poängen på A-delen uppnåtts. För betyg C krävs totalt minst 70 % av totala poängen; för betyg A krävs minst 90 % av totala poängen.
Övrig information:	Del A, Teoriuppgifter: Svara på separat papper. Del B, Programmeringsuppgifter: Lösningar lämnas på tentamenskontot. Alla lösningar skrivs i ett och samma projekt, med varje uppgift i ett separat paket, med namnen uppg9, uppg10, ... <i>När du skapar projektet i IntelliJ måste du kontrollera att projektet hamnar under enheten H, på ditt tentamenskonto.</i>

Mac-användare: För att komma åt måsvingar, hakparenteser och "|", använd "AltGr" + lämplig tangent, t.ex. "AltGr" + "{" (7:ans tangent).

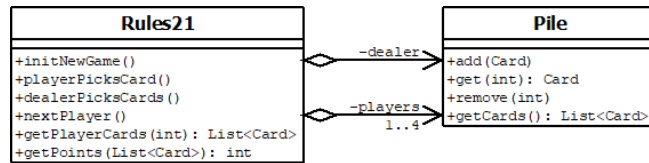
Del A, Teoriuppgifter

Svara, på separat papper, kort men koncist på frågorna. På fritextfrågor ska svaren motiveras. Otydliga lösningar, både vad gäller läsbarhet och begriplighet, rättas inte.

I uppgift 1–4 ska du ange vilka alternativ, noll till flera, som är korrekta. Svaren kan ges på ett gemensamt blad.

1. Vilka av nedanstående påståenden om nyckelordet final är korrekta i språket Java?
 - A. En datamedlem deklarerad final kan endast tilldelas ett värde en enda gång
 - B. En metod deklarerad final kan inte omdefinieras (override) i en subklass
 - C. En klass deklarerad final får inte sub-klassas (ärvas)
 - D. En lokal variabel kan alltid ändras, final har ingen påverkan i detta fall(2 p)
2. Vilka av nedanstående påståenden beskriver egenskaper hos, eller fördelar med, hög kohesion?
 - A. En klass ska modellera en och endast en sak
 - B. Att programmera mot interface är ett exempel på hög kohesion
 - C. Hög kohesion gör det enklare att testa klasser
 - D. Hög kohesion medför att ändringar i en del av en större kodbas har minimal påverkan på andra delar i koden
 - E. Inkapsling bidrar per automatik till hög kohesion(2 p)
3. Vilka av nedanstående påståenden om undantagshantering är korrekta?
 - A. Om ett undantag (exception) kastas i ett try-block avbryts exekveringen av blocket
 - B. Det får finnas inget till flera catch-block efter ett try-block och flera av dessa kan exekveras när undantag kastats
 - C. Ett finally-block exekveras endast om ett undantag kastats, men oberoende av vilken typ av exception som kastades
 - D. Det räcker i de flesta fall att skriva ut ett felmeddelande i ett catch-block för att felet ska kunna anses vara hanterat(2 p)
4. Vilka av nedanstående påståenden är korrekta för en abstrakt klass?
 - A. Det är tillåtet att skapa referensvariabler för en abstrakt typ
 - B. Det är tillåtet att skapa objekt av abstrakt typ
 - C. En abstrakt klass får ha metoder med implementation
 - D. Det är tillåtet att skriva en konstruktor i en abstrakt klass
 - E. En abstrakt klass måste innehålla minst en abstrakt metod(2 p)
5. Förklara begreppet polymorfism i objektorienterad programmering. Vad måste gälla för de ingående klasserna/metoderna för att vi ska kunna ha polymorfism?
(2 p)

6. Klassdiagrammet nedan beskriver en del av en applikation för ett kortspel.



- a) Förklara vad respektive symbol mellan Rules21 och Pile innebär, inklusive typ av relation, riktning, namn och siffror.
- b) Förklara vad symbolerna betyder innebär för implementationen av klassen Rules21.
- (2 p)
7. a) Beskriv designmönstret Facade, fasad.
- b) Facade är ett exempel på låg koppling – förklara varför.
- (2 p)
8. För att sortera en lista med textsträngar efter stigande längd kan man använda sig av metoden `java.util.Collections.sort` på nedanstående sätt.
- ```
Collections.sort(strList, (s1, s2) -> s1.length() - s2.length());
```
- Förklara vad det andra argumentet, i blått, som skickas till metoden `sort` representerar.
- (2 p)

## Del B, Programmeringsuppgifter

Lösningar lämnas i ett gemensamt projekt på tentamenskontot. För varje lösning ska det finnas ett paket med namnet "uppg9", "uppg10" et cetera. Till dessa uppgifter kan du i vissa fall ha hjälp av Javas API-dokumentation, som finns på `P:/Java/api`.

*Alla klasser i denna del av tentamen ska följa objektorienterade principer, speciellt inkapsling. Om inte annat anges ska alla klasser ha meningsfulla konstruktörer, som korrekt initierar alla datamedlemmar, access-metoder samt en omdefinierad version av `toString`.*

### 9. Vektor

Skriv en klass, `Vector2D`, som representerar en två-dimensionell vektor, med datamedlemmarna `x` och `y` (double). Objekt av typen `Vector2D` ska vara "immutable", dvs inte kunna ändra tillstånd efter att de skapats.

Klassen ska, förutom accessmetoder och `toString`, ha funktionaliteten nedan.

- En konstruktor med två argument, samt en parameterlös konstruktor (sätter 0,0).
- Metoden `length` som returnerar vektorns längd,  $\sqrt{x^2 + y^2}$  (se API-klassen `java.lang.Math`).
- Metoden `scale(double factor)` som returnerar ett nytt objekt där `x` och `y` multiplicerats med angiven faktor.
- Två metoder som adderar två vektorer och returnerar ett nytt objekt som representerar summan av vektorerna. Den första versionen tar en annan vektor som argument och adderar denna med `this`. Den andra versionen ska vara en klassmetod som tar två objekt som argument och returnerar summan av dessa. Vid summering av vektorer summeras `x`-respektive `y`-komponenter var för sig.
- En omdefiniering av metoden `equals` från klassen `Object`. Metoden ska returnera `true` om, och endast om, skillnaderna i `x`-värden och skillnaden i `y`-värden är mindre än en tusendel.

Skriv en main-metod som demonstrerar funktionaliteten.

(4 p)

## 10. Meddelanden

Det finns olika typer av meddelanden, bl.a. SMS och MMS. SMS är ett textmeddelande på max 160 tecken, ett MMS har en bild samt en kortare text.

Skriv en abstrakt klass, `AbstractMsg` med privata datamedlemmar för avsändare och mottagare (textsträngar, ska inte kunna ändras) samt en datamedlem för själva meddelandet, som endast ska vara synlig i subclasser. Klassen ska ha en abstrakt metod, `boolean setMessage(String)`.

Skriv därefter nedanstående subclasser.

- `SmsMsg`, som omdefinierar `setMessage` så att meddelandet kortas till 160 tecken om det är för långt. Om meddelandet kortats ska metoden returnera `false`.
- `MmsMsg`, som har en ytterligare datamedlem, som representerar en url till bilden (textsträng, ska inte kunna ändras). Konstruktorn ska kontrollera att bildens url börjar med `"https://"`, i annat fall ska ett `IllegalArgumentException` kastas. Metoden `setMessage` ska omdefinieras så meddelandet kortas till 80 tecken om det är för långt. Om meddelandet kortats ska metoden returnera `false`.

Alla klasser ska, förutom det som anges ovan, ha accessmetoder och en omdefinierad `toString`. Skriv sedan en main-metod som skapar objekt av subtyperna, lägger dessa i en gemensam lista, itererar över dessa och anropar någon av metoderna polymorft.

(4 p)

## 11. Lista

I denna uppgift ska du skriva en generisk list-klass som internt representeras med en array. Arrayen ska automatiskt utökas om den blir full.

Klassen ska implementera interfacet `IList`, filen kan kopieras tentamenskontot P/HI1027, med metoderna `isEmpty`, `get`, `addLast`, `clear`, och `copy`. Kontraktet för respektive metod framgår av dokumentationen i interfacet.

Klassen ska dessutom ha två konstruktörer, en där man kan ange en initial kapacitet samt en som är parameterlös.

(4 p)

## 12. Grafik

I denna uppgift kommer du att implementera en variant av designmönstret `Composite`, där förenklade versioner av grafiska objekt hanteras.

Skriv ett interface för grafiska objekt, `GraphicsObj`, som deklarerar de tre metoderna:

- `double area()`
- `void translate ( double dx, double dy )`
- `boolean isInside( double x, double y )`

Metoden `translate` flyttar figuren `dx` resp. `dy` i `x` resp. `y`-led. Metoden `inside` returnerar `true` om `x` och `y` ligger inom figuren.

Klasserna `Circle` och `Group` ska båda implementera interfacet `GraphicsObj` enligt nedan.

- Objekt av typen `Circle` har en mittpunkt  $(x, y)$  samt en radie.
- Objekt av klassen `Group` innehåller en mängd grafiska objekt, dvs. klasser som implementerar interfacet ovan. Metoden `area` returnerar i detta fall summan av alla areor i gruppen. Metoden `isInside` returnerar i detta fall `true` om punkten ligger inom någon av figurerna.

Det ska finnas en metod, `add`, för att lägga till grafiska objekt till gruppen, men en kontroll ska göras så att man inte kan lägga till objektet självt. Det ska även finnas en metod `remove(double x, double y)` som tar bort alla objekt ur gruppen där `x` och `y` ligger inom figuren.

Båda klasserna ska dessutom ha lämpliga accessmetoder samt en omdefinierad version av `toString`.

Skriv en `main`-metod som skapar en lista för `GraphicsObj`-objekt. Lägg till både cirklar och grupp-objekt i denna lista och demonstrera funktionaliteten, speciellt att lägga till objekt i en grupp.

(4 p)

```

/**
 * Functionality for an expandable and generic list.
 *
 * @param <E> the type of the elements in the list
 */
public interface IList<E> {
 /**
 * Return true if this list contains no elements.
 *
 * @return true if this list contains no elements
 */
 boolean isEmpty();

 /**
 * Appends the specified element to the end of this list.
 * Expand the internal store of elements if needed.
 *
 * @param element the element to append to this list
 * @throws NullPointerException if the specified element is null and this
 * list does not permit null elements
 */
 void addLast(E element);

 /**
 * Remove all elements from the list.
 * The implementation should indicate that the removed elements might be
 * eligible for garbage collection.
 */
 void clear();

 /**
 * Returns the element at the specified position in this list.
 *
 * @return the element at the specified position in this list
 * @throws IndexOutOfBoundsException if the index is out of bounds
 * @parameter index the index of the element to return
 */
 E get(int index);

 /**
 * Return a new list, a shallow copy, with the elements in this list.
 *
 * @return a shallow copy of the list.
 */
 IList<E> copy();
}

```